Experiment 3 — Optimising code generation

Mathilda — execution-speed experiments

Contents

Experiment 3 — Optimising code generation

Date: 2026-07-27 · Commits: `acb5210` (optimiser), `399938f` (CSE), `d5370a1` (frames/OP_CALL), `767f3a4` (K_BINK), `a99d3ec` (lazy operands), `0292153` (threaded dispatch) · Code: `src/compile/` · Result: ~1.5× on top of the base VM

Common method in `README.md`.

Hypothesis

Once a body compiles, the remaining cost is instructions retired per element. Standard compiler transforms should apply: constant folding, common subexpression elimination, dead code elimination, loop-invariant code motion, copy propagation. The question was how much they are worth on a bytecode VM, where each instruction already costs a dispatch.

What was built

A bytecode optimiser (`acb5210`): folding, CSE, copy propagation, DCE, LICM.

Expr-level CSE (`399938f`). The first CSE pass ran on the bytecode and barely fired, because by then the shared structure had already been lowered into different instruction sequences. Moving it up to the `Expr` tree — where `x y` in `x y + Sin[x y]` is literally the same subtree — made it work: 37.0 → 25.0 ns/call (1.48×).

That is the finding worth carrying: CSE has to run where the expression is still an expression. A peephole pass over instructions cannot recover information the lowering has already destroyed.

Constant operands folded into the instruction (`K_BINK`, `767f3a4`). A literal in the body was emitted as a separate `OP_CONST` push; for a short body that was a third of the entire stream. `Compile[{{x,_Real}}, 1.5 + 2.5 x]` went from 5 instructions to 3.

body	instructions	ns/call
Horner, degree 40	121 → 81 (−33%)	181 → 173 (1.05×)
`1.5 + 2.5 x`	5 → 3 (−40%)	
`Table[degree-5 poly, 10⁶ points]`		140 → 129 ms (1.09×)
Newton (`While`) micro-bench	20 → 14	382 → 340 ns (1.12×)

Note the honest shape of that table: a 33% instruction-count reduction buys 5% on Horner. Instruction count is not time — the folded operands were the cheapest instructions in the stream. The win is real but small, and reporting the instruction count alone would have overstated it by 6×.

Per-call frames and **`OP_CALL`** (`d5370a1`): a compiled function can call another compiled function without going back through the interpreter.

Threaded (computed-goto) dispatch (`0292153`) and lazily addressed operands (`a99d3ec`).

Results

The optimiser's contribution, measured within Mathilda at −03:

	before	after	
Horner, degree 40	295 ns/call	~200 ns/call	~1.45×
Expr-level CSE on `x y + Sin[x y]`	37.0 ns/call	25.0 ns/call	1.48×
Global constant folding alone			1.33× (Horner), 1.29× (`Nest`)

End-to-end, against the other two systems, on the kernels this affects:

Benchmark	Mathilda	Mathematica 14.0	NumPy / Python	vs WL	vs NumPy
Logistic map, 10^7 iterations	183.76 ms	254.36 ms	460.57 ms	1.38x	2.51x
Mandelbrot, 800x800, 100 iterations	769.23 ms	1.621 s	980.20 ms	2.11x	1.27x
Lennard-Jones energy, 1452 bodies (all pairs)	133.19 ms	319.51 ms	91.25 ms	2.40x	1/1.46x
Monte Carlo pi, 10^7 samples (vectorized)	162.36 ms	250.40 ms	204.53 ms	1.54x	1.26x

The Python column is vectorised NumPy for Lennard-Jones, Mandelbrot and Monte Carlo π — so those three are genuine library comparisons — and a CPython loop for the logistic map, which is a scalar recurrence with no vectorised form (numba is not installed on this host). Only the logistic-map row should be read as "what a Python user without a JIT measures".

The **`-O0`** trap

The first measurement of this work reported 1.53× on Horner — at −00. At −03 the same change measured essentially nothing, because gcc was already performing the equivalent transforms on the VM's own C code. The figures were re-measured and the changelog corrected in `cb3d978`.

This is the second appearance of the same class of error in these experiments (the first is in `COMPILE_ARRAY_FUSION.md`), and the rule that came out of it is in `tasks/lessons.md`:

> Never benchmark a compiler optimisation against a debug build. The question is whether your transform helps the shipped binary, and at −00 you are measuring how much of the C compiler's job you have duplicated.

Why the ceiling is low

~1.5× is a modest return for five classical optimisations, and the reason is structural: a bytecode VM's floor is dispatch, not arithmetic. Removing an instruction removes one dispatch; removing a redundant `Sin` removes a libm call. Only the second is large, which is why CSE (1.48×) beat operand folding (1.05×) despite folding removing more instructions.

Getting past this ceiling means emitting native code rather than bytecode, which is a different project with a very different dependency footprint. It is not planned.

Still open

- No register allocation across basic blocks.
- No strength reduction on the index arithmetic in indexed-array bodies.
- The optimiser does not run on the fused array loop's generated stages, only on the scalar stream.

Why Mathilda is not the fastest here, and what it would take

This experiment has no benchmark of its own — it measures the optimiser against the unoptimised compiler, and the four application rows it reports are experiment 1's, where Mathilda leads Mathematica on all four and NumPy on three.

The honest finding is the one the write-up already states: the ceiling is low, 1.05–1.48×, because a bytecode VM's dispatch cost is already the dominant term and removing instructions helps only in proportion.

The road to fastest

The optimiser is not where the remaining performance is. In value order, what would actually move the compiled path:

1. Reduce dispatch, not instruction count. A computed-goto or tail-threaded interpreter loop typically buys 1.3–2× over a `switch`-based one, and it multiplies every kernel rather than the subset an optimiser can simplify. This is worth more than every optimisation pass in this experiment combined.
2. Superinstructions for the measured hot pairs — `load; multiply`, `multiply; add` — which is a smaller, safer version of the same idea and can be driven directly from a profile of the four application kernels.
3. Native code generation for the straight-line numeric subset. This is the only step that removes dispatch entirely, and it is a large piece of work with a large payoff; it should not be started until 1 and 2 have established what the dispatch floor actually is.

Item 1 is the one to do first, and this experiment is what says so: an optimiser that removes a third of the instructions bought 5%.